

LAMBADA & MMLU Benchmark Evaluation Report

Evaluating small language models on long-range word prediction (LAMBADA) and multi-subject knowledge & reasoning (MMLU) — architecture, fine-tuning parameters, metrics, and results explained in full.

Submitted to: Prof. Anna Corazza

Submitted by: Francesco Ventimiglia, Danilo Rodriguez, Rohan Baidya

Github link: <https://github.com/ronvoy/gen-ai>

Site Link: <https://unina.cc/gen-ai>

1. Research Question

Among three small language models built with different compression strategies and attention mechanisms, which one best balances accuracy, reasoning quality, and latency — on long-range context prediction (LAMBADA) and broad knowledge & reasoning (MMLU)?

In particular: does a larger, non-distilled model (Ministral-8B) outperform smaller distilled/pruned models (Gemma-3-4B, Llama-3.2-3B) across both benchmarks — or does compression cost more in one dimension (accuracy, reasoning, speed) than another?

2. Experimental Protocol

Dataset

- LAMBADA: 5,153-passage test split from BookCorpus (English, ACL 2016).
- MMLU: 57 subjects × 5 questions/subject = 285 questions per model, fetched from the free HF datasets-server API and cached locally.

Preprocessing

- LAMBADA: predictions normalized (lowercase, punctuation stripped) before exact-match comparison.
- MMLU: chain-of-thought prompt with one worked example enforcing a strict Reasoning: / Answer: <letter> format.

Decoding setup (no training)

- No model weights are changed — only decoding parameters (temperature, top-p, max tokens, few-shot count) vary.
- Shared presets — Optimal, Normal, Best Performance — apply identically to both benchmarks and all three models for a like-for-like comparison.

Evaluation strategy

- All models queried through the OpenRouter API for a hardware-neutral comparison.
- Deterministic sampling — fixed subject/question order, no randomness — for reproducible runs.
- Metrics captured live in one shared web app: terminal streaming, run history, charts.

3. Benchmark

Baseline task

LAMBADA: predict the single next word of a passage, guessable only from full context. MMLU: choose the correct option (A–D) after writing short step-by-step reasoning. No fine-tuning either way — every model is evaluated purely on its pretrained + instruction-tuned weights.

Reference criteria — LAMBADA

Metric	Better
Exact-match accuracy	Higher
Average response time	Lower
Error rate	Lower
Throughput	Higher

Reference criteria — MMLU

Metric	Definition
Accuracy	Correct letters / total (exact match)
Category accuracy	Per STEM / Humanities / Social Sciences / Other
Reasoning consistency	Reasoning supports the chosen option
Composite score	0.70·accuracy + 0.15·reasoning + 0.15·speed

Ground truth for both benchmarks comes directly from the source datasets (the target word / the official answer key) — never inferred.

4. Involved Approaches

#	Model	Developer	Parameters	OpenRouter id	Architecture	Key Technique
1	Gemma-3-4B	Google	4B	google/gemma-3-4b-it	Dense decoder-only transformer with interleaved local/global attention	Knowledge distillation + local/global attention interleaving
2	Llama-3.2-3B	Meta	3B	meta-llama/llama-3.2-3b-instruct	Dense decoder-only transformer with Grouped	Compact dense transformer

#	Model	Developer	Parameters	OpenRouter id	Architecture	Key Technique
					Query Attention	
3	Ministral-8B	Mistral AI	8B	mistralai/ministral-8b-2512	Decoder-only transformer with Sliding Window Attention	Sliding Window Attention + GQA

- Gemma-3-4B: dense decoder-only transformer with interleaved local/global attention, distilled from a larger teacher, 128k-token context.
- Llama-3.2-3B: standard dense transformer recipe — RoPE, Grouped Query Attention, SwiGLU — pruned and distilled from larger Llama 3.1 models.
- Ministral-8B: sliding window attention + GQA for efficient long-context inference at the edge — trained at its native 8B size, not distilled down from a larger checkpoint.

5. Considered Model(s)

5.1 Gemma-3-4B (Google)

- 4B instruction-tuned dense decoder-only transformer from Google, built with the same research that powers Gemini.
- Interleaves several local sliding-window attention layers with an occasional global attention layer, keeping memory low while information still flows across a long context (up to 128k tokens).
- The small Gemma 3 models are trained with knowledge distillation from larger teacher models.

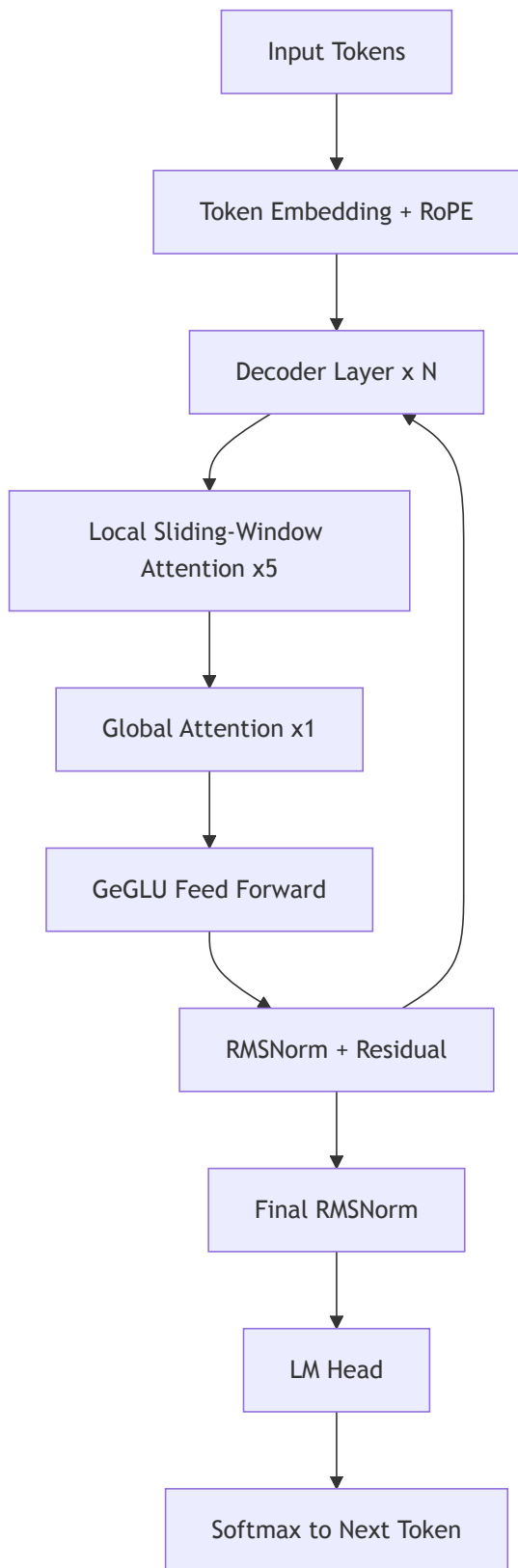
Working technique - Knowledge distillation + local/global attention interleaving:

- Most layers only look at a local window of nearby tokens (cheap); every few layers one global layer lets any token attend to the whole context (expressive).
- Distillation teaches the small model to match a larger teacher's output distribution rather than raw text alone, delivering strong quality per parameter at edge-friendly cost.

Key properties:

- Dense decoder-only transformer, no expert routing.
- 5:1 interleaving of local sliding-window and global attention layers.
- Grouped Query Attention with QK-norm; 128k-token context window.
- Distilled from larger Gemma/Gemini-family teacher models.

Process flow - Gemma-3-4B inference



- Local sliding-window attention x5: cheap, restricted to nearby tokens.
- Global attention x1: the one layer per block where every token can see the full passage — this is how long-range context (LAMBADA's whole point) actually gets used.
- GeGLU feed forward: a gated variant of the MLP block used after attention.

5.2 Llama-3.2-3B (Meta)

- 3B instruction-tuned model from Meta for on-device, low-cost use.
- Standard Llama recipe: dense transformer with RoPE, GQA, and SwiGLU layers.
- Built by pruning and distilling from larger Llama 3.1 models.

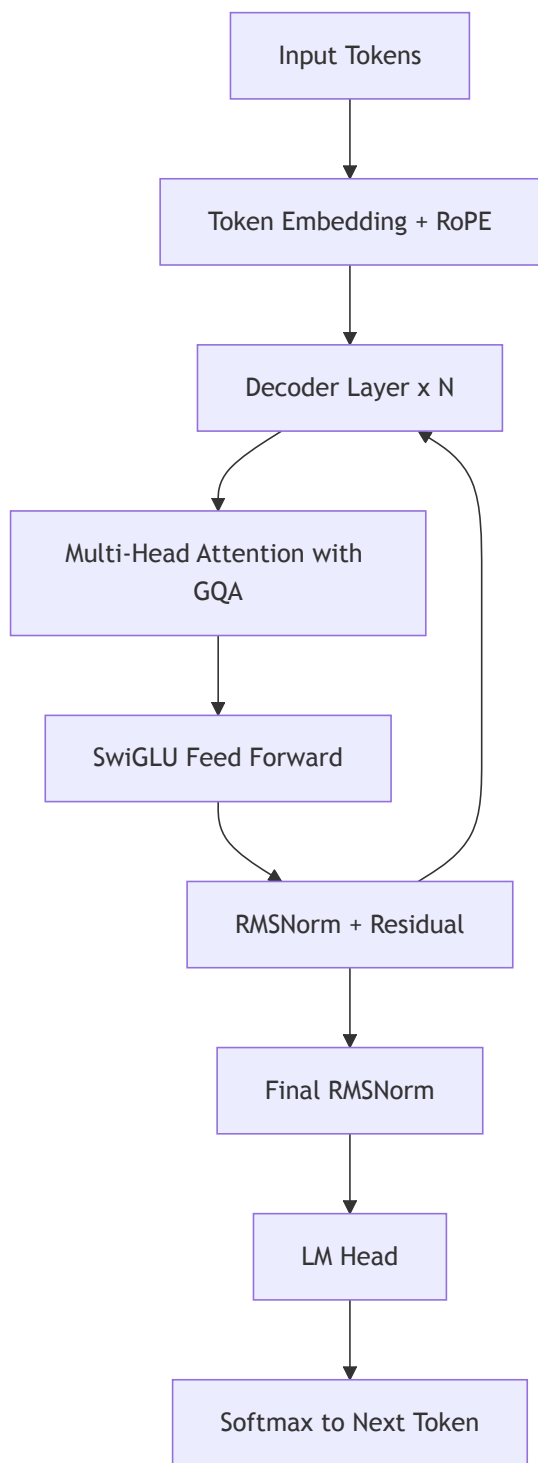
Working technique - Compact dense transformer:

- Takes the proven dense transformer design and shrinks it — no architectural novelty, the standard Llama recipe simply scaled down to 3B parameters rather than a smaller model with new efficiency tricks.
- Predictable and easy to serve on modest hardware.

Key properties:

- Dense decoder-only transformer.
- RoPE position embeddings for length generalisation.
- Grouped Query Attention and SwiGLU layers.

Process flow - Llama-3.2-3B inference



- Multi-head attention (GQA): full attention over the whole context, but with grouped key/value heads to shrink the KV cache.
- SwiGLU feed forward: a gated MLP block, used in place of a plain ReLU/Gelu MLP.
- No local/global split, no sliding window — the simplest of the three designs.

5.3 Ministral-8B (Mistral AI)

- 8B model from Mistral AI, built for edge use.
- Interleaved sliding window attention keeps memory and compute low on long inputs.
- Grouped query attention further shrinks the KV cache.

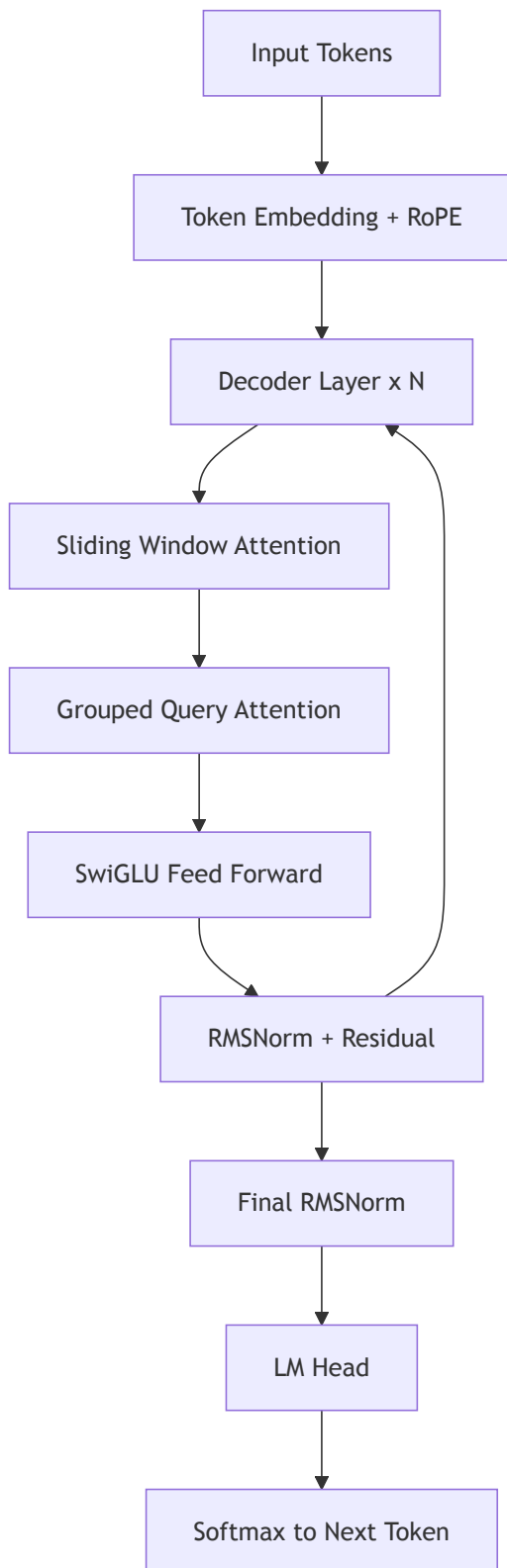
Working technique - Sliding Window Attention + GQA:

- Each layer attends only to a local window instead of every token.
- Stacked layers carry context further than any single window.
- GQA shares key-value heads for fast decoding on long passages.
- Unlike the other two models in this study, Ministral-8B is not described as distilled or pruned from a larger checkpoint — it is trained at its native 8B size, very likely the single biggest reason it leads both benchmarks (see Section 8).

Key properties:

- Sliding window attention for local context.
- Grouped Query Attention for a smaller KV cache.
- Efficient long-context inference at the edge.

Process flow - Ministral-8B inference



- Sliding window attention: local context only, per layer — same trick Gemma uses for its "local" layers, but used on every layer here, not just 4 in 5.
- Grouped Query Attention: shrinks the KV cache for fast decoding on long passages.
- No global-attention layer at all — depth (stacking many windowed layers) substitutes for it.

6. Comments and Discussion before Results

This section walks through every fine-tuning parameter, preset, metric, dataset detail, and workflow used in this study — the "how and why" behind the numbers in Section 7.

6.1 Fine-Tuning Parameters, explained

The Fine Tune panel exposes six decoding parameters, shared by both benchmarks. None of them change model weights — they only control how the next token is sampled at inference time.

Temperature (range 0.0 – 2.0, used here: **0.0**)

Scales the logits before the softmax that turns the model's raw output scores into a probability distribution over the vocabulary — the next token is then sampled from that distribution. At 0.0 the distribution collapses to argmax — the single highest-probability token is always chosen (greedy decoding): fully deterministic, same input always gives the same output. Higher values flatten the distribution, letting lower-probability tokens get picked sometimes — more variety, but more risk of drifting off the one correct word or letter. Both LAMBADA (one correct word) and MMLU (one correct letter) have exactly one right answer — no reward for creative variation, only downside risk. This is why every model in this study, regardless of size or architecture, was run at temperature 0.0.

Top-p / nucleus sampling (range 0.0 – 1.0, used here: **1.0**)

Restricts sampling to the smallest set of tokens whose cumulative probability reaches p — the "nucleus". Anything outside that set is discarded before a token is sampled. At 1.0 the nucleus includes the entire distribution — no tokens are excluded. Combined with temperature 0.0, top-p has no practical effect here: greedy decoding already picks one token deterministically regardless of how large the candidate pool is. Lower values would only matter at temperature > 0, where they'd narrow sampling to the most confident tokens. It is set to its neutral value here rather than doing any real work in this evaluation.

Max tokens (range 1 – 128 on the LAMBADA panel, used here: **32 for LAMBADA, 384 for MMLU**)

A hard cap on how many tokens the model is allowed to generate before the API cuts it off. Every output token is one more sequential decoding step, so this parameter is also a direct lever on latency, not just on answer length. LAMBADA needs only a single word plus a little slack for stray punctuation or formatting — 32 tokens is generous headroom. MMLU needs far more: the model must write a full chain-of-thought explanation before its final "Answer: " line. Setting it too low truncates the model mid-reasoning before it reaches the answer line — recorded as a "no-answer" verdict, not a wrong answer. This budget difference is a major reason MMLU latency (0.6–2.5s) runs systematically higher than LAMBADA latency (0.6–1.2s) for the same models.

Few-shot examples (range 0 – 5 on LAMBADA, used here: **3 worked examples**)

Prepends worked (context → answer) examples from a fixed pool before the real passage, showing the model the exact input/output format expected — a prompting technique, not a change to the model's weights. Instruction-tuned models sometimes answer in full sentences ("The next word is likely...") unless shown the expected bare-word format; a few worked examples anchor that format without touching the model's weights. MMLU instead uses one fixed worked example baked into the prompt template, rather than a tunable count. Trade-off: each example adds prompt tokens, so more few-shot examples cost a little extra latency per call, even though the model's own output doesn't get any longer.

Frequency penalty (range -2.0 – 2.0, used here: **0.0**)

Subtracts a penalty proportional to how many times a token has already appeared in the output so far — the more it repeats, the less likely it is to repeat again.

Presence penalty (range -2.0 – 2.0, used here: **0.0**)

Applies a flat penalty to any token the moment it appears at least once, regardless of count — encourages introducing new words/topics rather than staying on the same ones.

Both penalties are set to neutral (0.0) for every model, on both benchmarks: both tasks produce a single short answer — one word, or one letter plus brief reasoning — so there's no room for the kind of repetitive looping these penalties are designed to prevent in long-form, open-ended generation.

6.2 Parameter Set Used in This Evaluation

Parameter	LAMBADA	MMLU
Temperature	0.0	0.0
Top-p	1.0	1.0
Max tokens	32	384
Frequency penalty	0.0	—
Presence penalty	0.0	—
Few-shot examples	3	—

This is a hand-set configuration close to, but not identical to, any single named preset (it mixes Optimal's greedy decoding with a different max-token / few-shot budget). It was applied identically to Gemma-3-4B, Llama-3.2-3B, and Ministral-8B — none of the three received special treatment, such as a larger reasoning-token budget. `config.py` reserves a separate `MAX_TOKENS_REASONING = 8096` budget for models flagged in `REASONING_MODELS` — that list is empty for this study, so none of the three models used it. Any differences in the results in Section 7 come from the models themselves, not from unequal evaluation conditions.

6.3 Fine-Tuning Presets

Both benchmark pages expose one-click presets over the decoding parameters. They control decoding, not model weights.

LAMBADA Presets (PRESETS in [config.py](#))

Preset	Temperature	Top-p	Max tokens	Few-shot	Intent
Optimal	0.0	1.0	16	5	Greedy decoding with the most worked examples - most reliable accuracy
Normal	0.3	0.9	32	3	Balanced default
Best Performance	0.0	1.0	8	2	Trimmed token and example budget - fastest, cheapest runs

Optimal is theoretically the strongest preset for LAMBADA specifically: the task has exactly one correct token, so greedy decoding removes sampling risk entirely, and more worked examples further anchor the one-word output format. Normal reintroduces randomness (temp 0.3, top-p 0.9) — useful for exploring variability, but with no upside on an exact-match task like this one. Best Performance trims both the token cap and the example budget for the cheapest, fastest runs, at some risk of

losing the format anchor. The actual saved runs (Section 6.2) used a nearby but distinct hand-set configuration rather than one of these three presets exactly.

MMLU Presets (`MMLU_PRESETS` in [config.py](#))

Preset	Temperature	Top-p	Max tokens	Intent
Optimal	0.0	1.0	512	Most reasoning room - most reliable accuracy
Normal	0.2	0.95	384	Matches the defaults
Best Performance	0.0	1.0	192	Caps the reasoning budget so runs finish faster

Optimal gives the most room (512 tokens) for multi-step reasoning before the model commits to a letter — useful for subjects needing longer derivations (e.g. `formal_logic` , `college_mathematics`). Best Performance risks truncating longer reasoning chains before the model reaches its "Answer:" line, which would surface as a "no-answer" verdict. The actual saved runs used 384 tokens — matching Normal's budget — with greedy decoding (temperature 0.0, top-p 1.0) borrowed from Optimal.

6.4 Metrics Explained — LAMBADA

Metric	Explanation
Exact-match accuracy	correct / total, after lowercasing and stripping punctuation from both the prediction and the true target word. Example: target "Zane." vs. prediction "zane" — different strings, but identical after normalization → counted correct.
Average response time	Mean wall-clock seconds per API call, start to finish — not just model compute time. Includes network round-trip and OpenRouter's provider routing/queueing, which is why it can spike heavily under provider congestion.
Error rate	errors / total — API calls that failed outright (timeouts, malformed responses) even after retries were exhausted. All three models finished at 0 errors in the saved runs — every request eventually succeeded, even ones needing multiple retries.
Throughput	total / total wall-clock time — samples graded per second. Requests are sent one at a time here (no batching), so throughput is essentially the reciprocal of latency: Llama ≈1.73/s, Ministral ≈1.37/s, Gemma ≈0.84/s.

6.5 Metrics Explained — MMLU

Metric	Explanation
Accuracy	Correct letters / total questions (exact match) — judges only the final answer, not the reasoning behind it.
Category accuracy	The same accuracy computed separately per STEM / Humanities / Social Sciences / Other. Reveals blind spots the headline number hides — e.g. Llama-3.2-3B: 43.3% STEM vs. 63.3% Social Sciences, a 20-point spread.
Reasoning rate	Share of answers with a non-trivial explanation (≥5 words). All three models wrote substantive reasoning on effectively every question (≈100% reasoning rate) — the differentiator is never whether they explained, but whether the explanation held up.

Metric	Explanation
Reasoning consistency & avg. words	Consistency = share of answers whose reasoning text actually supports the chosen letter. Ministral-8B writes the shortest reasoning (54.1 words) yet the highest consistency (69.1%); Llama-3.2-3B writes the most (61.8 words) yet is least consistent (59.7%) — length and quality of explanation are not the same thing.

Composite score, worked example

Composite = 0.70 x accuracy + 0.15 x reasoning consistency + 0.15 x relative speed
Relative speed = fastest model's avg time / this model's avg time

Model	Accuracy term (0.70x)	Reasoning term (0.15x)	Speed term (0.15x)	Composite
Ministral-8B	$0.789 \times 0.70 = 0.552$	$0.691 \times 0.15 = 0.104$	$0.500 \times 0.15 = 0.075$	0.731
Llama-3.2-3B	$0.558 \times 0.70 = 0.391$	$0.597 \times 0.15 = 0.090$	$1.000 \times 0.15 = 0.150$	0.630
Gemma-3-4B	$0.639 \times 0.70 = 0.447$	$0.646 \times 0.15 = 0.097$	$0.307 \times 0.15 = 0.046$	0.590

The speed term uses Llama-3.2-3B's 0.768s as the fastest reference time: Ministral $0.768/1.537=0.500$, Llama $0.768/0.768=1.000$, Gemma $0.768/2.502=0.307$. These figures reconcile exactly with the composite scores reported in Section 7.3. Accuracy dominates the weighting (70%), so Ministral's large accuracy lead carries it to #1 even though it is not the fastest model.

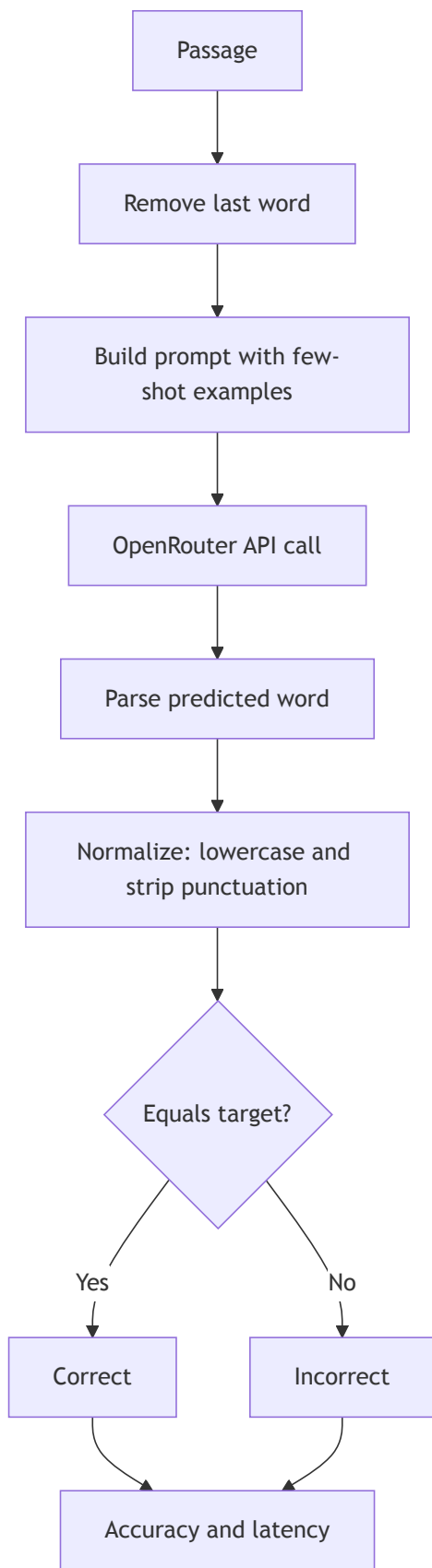
6.6 LAMBADA: Dataset, Properties & Scoring Flow

- LAMBADA is drawn from the BookCorpus.
- Each passage is chosen so the final word is predictable from the full passage but not from the last sentence alone.

Properties

Property	Value
Source corpus	BookCorpus (unpublished novels)
Language	English
Task type	Word prediction
Curation criterion	Target guessable from full context only
First published	ACL 2016 (Paperno et al.)

Scoring flow



Normalization is what makes exact-match accuracy fair — without it, capitalization or a trailing period would wrongly count a correct guess as wrong. Latency is recorded for every call — correct, incorrect, or (after retries) failed — so the average reflects real-world response time, not just successful guesses.

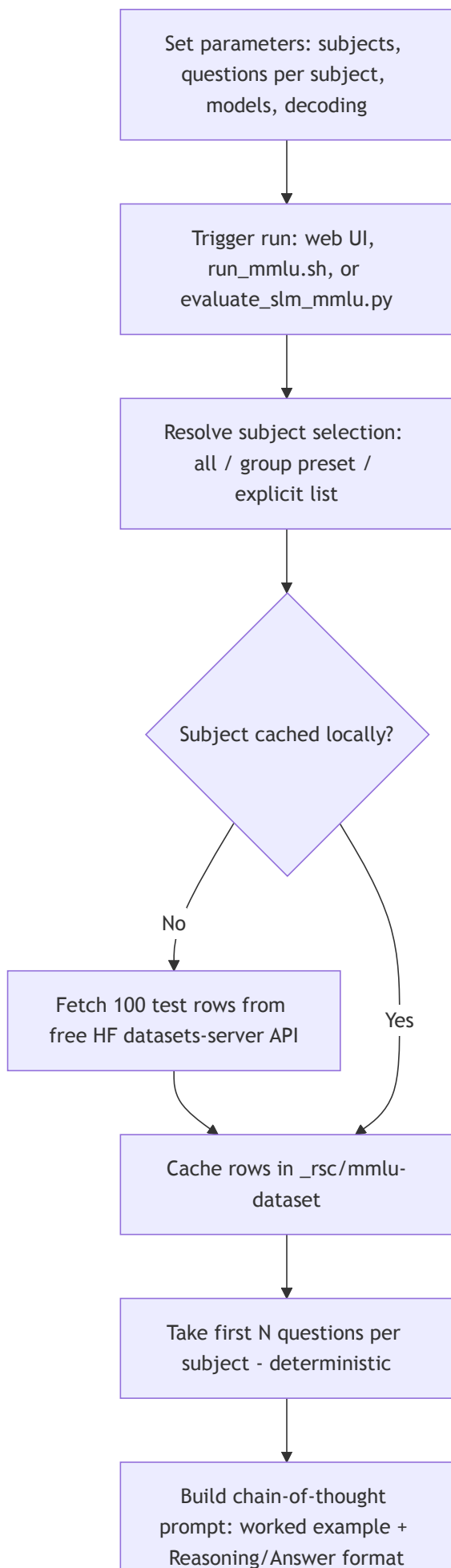
6.7 LAMBADA: Dataset Splits

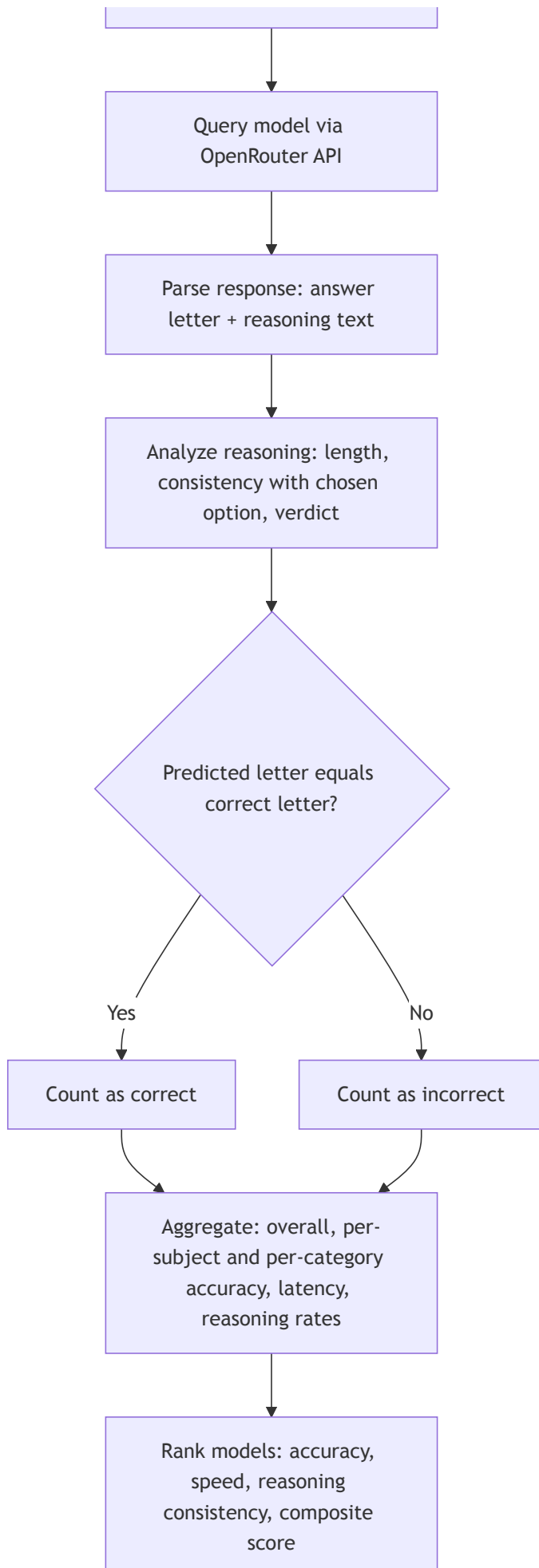
Split	File	Passages	Purpose
Test	lambada_test_plain_text.txt	5,153	Primary evaluation
Development	lambada_development_plain_text.txt	4,869	Validation and tuning
Control Test	lambada_control_test_data_plain_text.txt	5,000	Baseline, unfiltered
Rejected	rejected_plain_text.txt	11,941	Passages cut during curation
Training Novels	train-novels/ (16 genres)	2,662 novels	Pre-training material
Vocabulary	lambada-vocab-2.txt	112,746 entries	Reference vocabulary

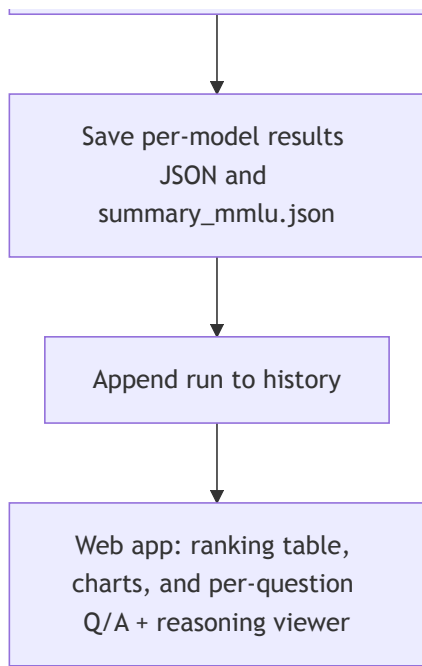
This evaluation uses only the Test split (5,153 passages, sampled down to 1,000 for the saved runs) — the other splits exist in the released dataset but aren't used for scoring here.

6.8 MMLU: How a Run Works

How the MMLU benchmark runs in this project, from a click on "Run MMLU" (or `./run_mmlu.sh`) to the ranking table, charts, and per-question reasoning shown in the web app. For every question the model must first write short step-by-step reasoning and then commit to one of four options (A-D); both the final letter and the reasoning text are parsed and evaluated.







Chronological steps

#	Step	What happens	Where (file / function)
1	Parameters	Read subjects, questions per subject, models, decoding params (all documented at the top of the script and shell runner)	evaluate_slm_mmlu.py (RUN PARAMETERS), run_mmlu.sh
2	Trigger	User clicks Run MMLU, runs <code>./run_mmlu.sh</code> , or <code>python evaluate_slm_mmlu.py</code>	passenger_wsgi.py (/mmlu/run) or run_mmlu_evaluation
3	Resolve subjects	Turn "all", a group preset, or a list into valid subject names	resolve_subjects
4	Fetch questions	Download the first 100 test rows per subject from the free Hugging Face datasets-server API (cais/mmlu, fallback tasksource/mmlu); no API key needed	_download_subject
5	Cache	Store rows in <code>_rsc/mmlu-dataset/<subject>.json</code> so reruns are offline and repeatable	fetch_subject_questions
6	Sample	Take the first N questions per subject (deterministic, no randomness)	fetch_subject_questions , load_mmlu_tasks
7	Build prompt	Chain-of-thought prompt: one worked example, strict Reasoning: then Answer: <letter> output format	build_mmlu_prompt , WORKED_EXAMPLE
8	Query model	Send the prompt to the chosen model through OpenRouter	query_model_mmlu (HTTP POST)
9	Parse	Extract the answer letter AND the reasoning text (handles Reasoning:/Answer: , <think> blocks, "the answer is (B)", bare letters, truncation)	parse_mmlu_response
10	Evaluate reasoning	Score the reasoning: is it present, how long, does it actually support the chosen option; assign a verdict	analyze_reasoning

#	Step	What happens	Where (file / function)
11	Compare	Exact match: predicted letter equals the correct letter	<code>evaluate_model_mmlu</code>
12	Aggregate	Overall / per-subject / per-category accuracy, latency, errors, reasoning rates	<code>evaluate_model_mmlu</code>
13	Rank	Per-dimension ranks and composite score across all evaluated models	<code>build_mmlu_summary</code>
14	Save	Write <code>results/<model>_mmlu.json</code> and <code>results/summary_mmlu.json</code>	<code>run_mmlu_evaluation</code> / <code>run_mmlu_benchmark</code>
15	History	Append the run (models, subjects, question count, params) to history	<code>passenger_wsgi.append_history</code>
16	Present	Ranking table, accuracy charts, and the per-question Q/A + reasoning accordion	<code>templates/mmlu.html</code> (Chart.js)

6.9 MMLU: Components

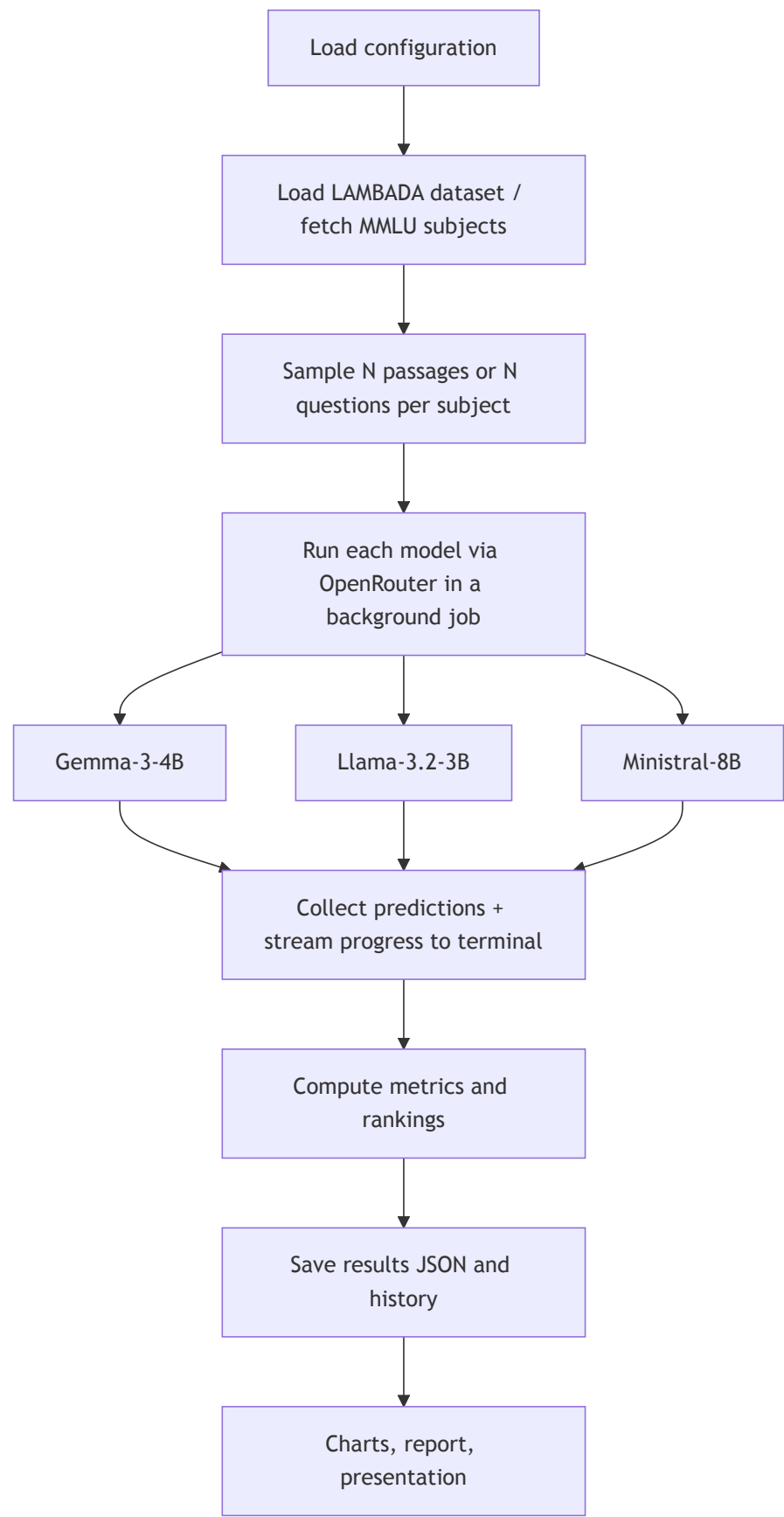
Component	File	Role
Run parameters	<code>evaluate_slm_mmlu.py</code> (top), <code>run_mmlu.sh</code> (top)	Subject selection ("all", group presets, or explicit list), questions per subject (1-100), models, decoding params - every option listed in comments
Subject catalogue	<code>MMLU_SUBJECTS</code> , <code>SUBJECT_GROUPS</code> , <code>CATEGORY_LABELS</code>	The 57 subjects mapped to their official categories; group presets derived from the mapping
Dataset fetcher	<code>fetch_subject_questions</code> , <code>_download_subject</code>	Free HF datasets-server API client with source fallback and local JSON cache
Task loader	<code>load_mmlu_tasks</code>	Flattens subjects x questions into one ordered task list
Prompt builder	<code>build_mmlu_prompt</code> , <code>WORKED_EXAMPLE</code>	Chain-of-thought prompt with a worked example enforcing a parseable format
Model client	<code>query_model_mmlu</code>	OpenRouter chat-completions call with timing and error capture
Response parser	<code>parse_mmlu_response</code>	Splits a raw response into (answer letter, reasoning text)
Reasoning analyzer	<code>analyze_reasoning</code>	Heuristic quality check: presence, word count, consistency, verdict
Evaluator	<code>evaluate_model_mmlu</code>	Runs all tasks for one model; aggregates accuracy and reasoning metrics
Ranker	<code>build_mmlu_summary</code>	Per-dimension ranks + composite score across models

Component	File	Role
CLI runner	run_mmlu_evaluation , _parse_cli , run_mmlu.sh	One-shot terminal pipeline: setup, install, run, print ranking
Web routes	passenger_wsgi.py : /mmlu , /mmlu/run , /mmlu/metrics , /mmlu/details , /progress/<job_id>	Online runs, live progress feed, ranking JSON, and the per-question detail feed
Web page	templates/mmlu.html	Subject picker with presets, decoding sliders + presets, live terminal, ranking table, charts, Q/A + reasoning viewer

6.10 MMLU: Reasoning Verdicts (per question)

Verdict	Meaning
sound	Correct answer, and the reasoning clearly supports it
right-weak-link	Correct answer, reasoning present but does not clearly support the pick
lucky-guess	Correct answer with no real reasoning
flawed-reasoning	Reasoned its way to a wrong answer
blind-guess	Wrong answer and no reasoning
no-answer	No A-D letter could be parsed (API error or malformed output)

6.11 Project Workflow — End to End



7. Results

7.1 LAMBADA — Results (test split)

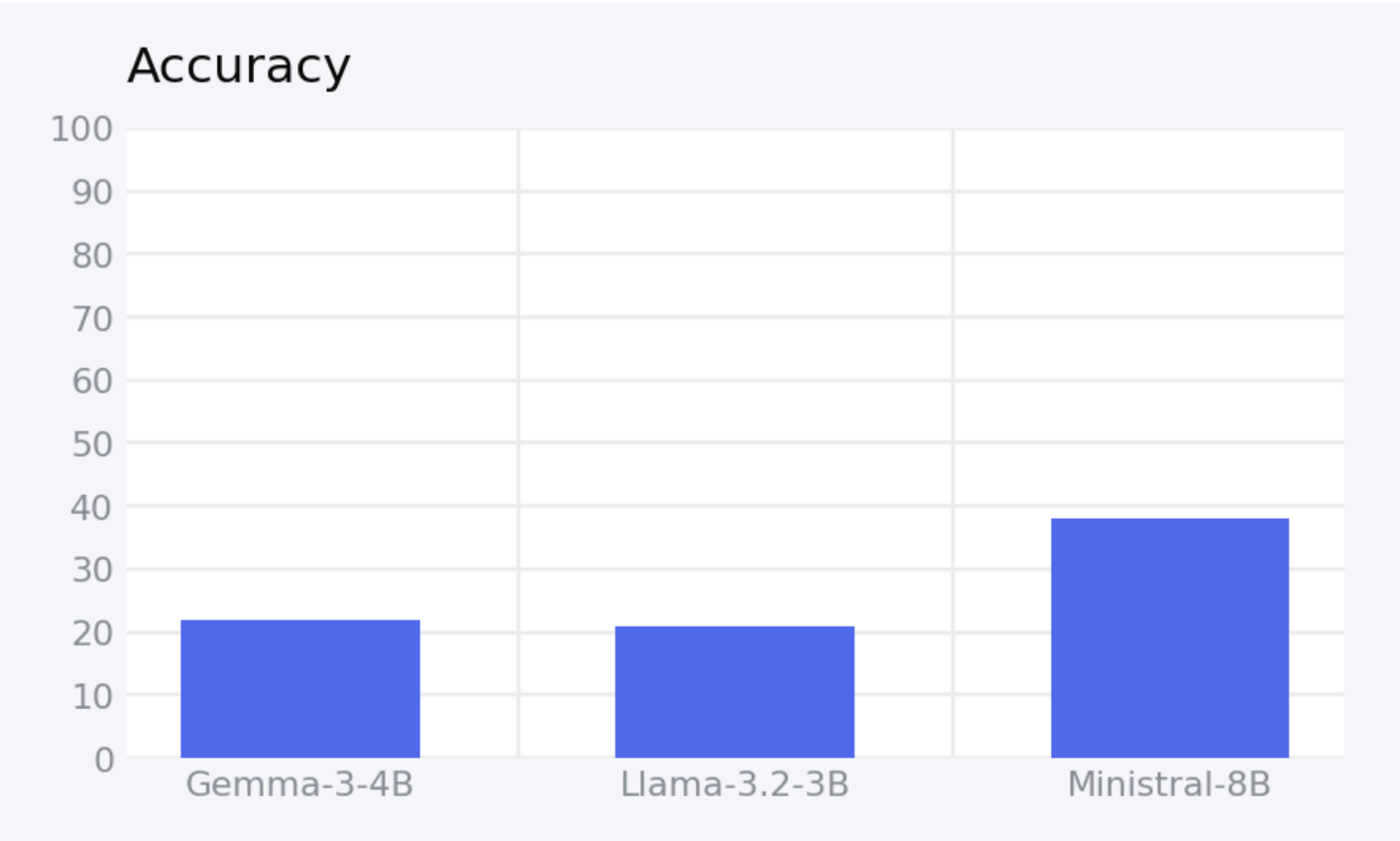
Latest saved metrics per model on the test split.

Model	Accuracy (%)	Correct	Total	Avg Latency (s)	Errors
Gemma-3-4B	21.9	219	1000	1.198	0
Llama-3.2-3B	20.8	208	1000	0.577	0
Ministral-8B	38.1	381	1000	0.731	0

Best accuracy: Ministral-8B at 38.1 percent. Fastest: Llama-3.2-3B at 0.577 s per query. Gemma-3-4B's accuracy holds steady at 21.9 percent on the full 1,000-sample test split (consistent with its earlier 50-sample estimate of 22.0 percent), but at this sample size it is now clearly the slowest model at 1.198 s per query, well behind Llama-3.2-3B and Ministral-8B.

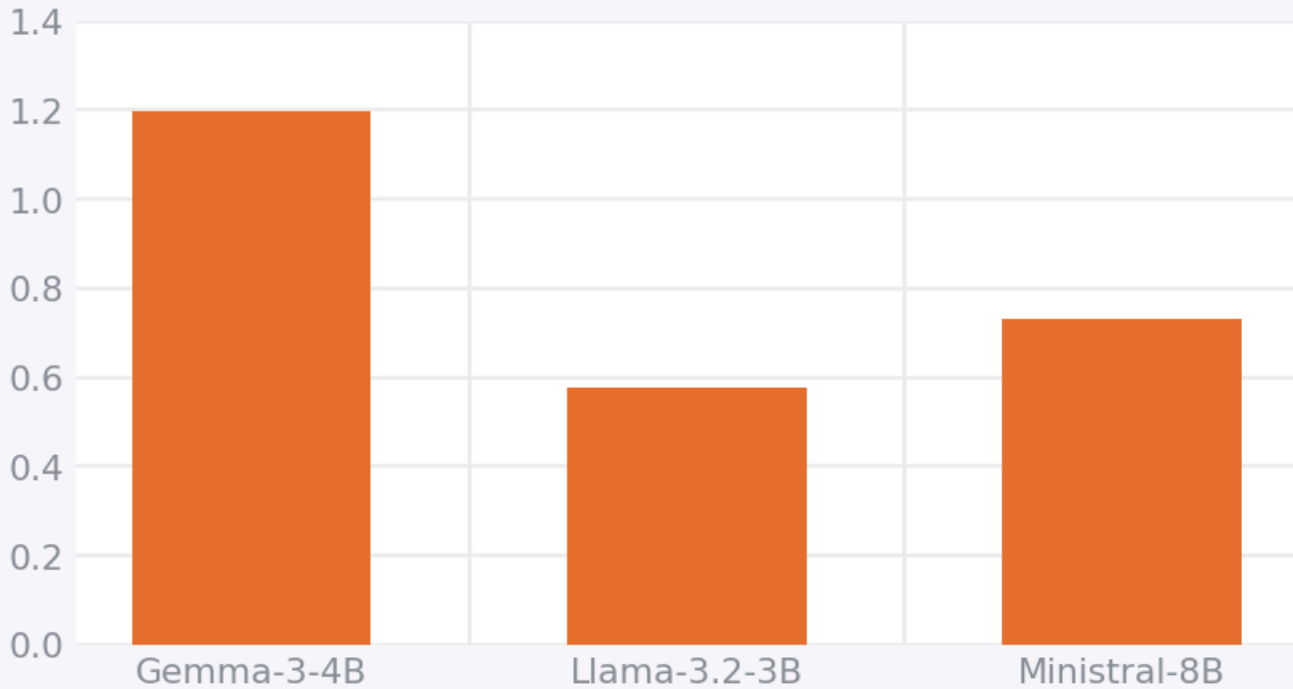
Charts

Accuracy per model:



Average response time per model:

Average response time (s)



7.2 LAMBADA — SLM Web App

The LAMBADA run panel: model and sample-count inputs, Run Benchmark and Fine Tune (presets) buttons, the live terminal streaming per-sample processing output (green = correct prediction, red = wrong), and the metrics table below with the latest saved results for all three models.

SLM Benchmark

LAMBADA MMLU History Docs

Small Language Model Benchmark

Pick a model and run the LAMBADA word-prediction task through the OpenRouter API. Results are saved to the results folder and shown below.

Model

Samples (1-1000)

Gemma-3-4B

1000

Run Benchmark

Fine Tune

History

● ● ● lambada — benchmark output

[12:38:04] [992/1000] x target="mugger" predicted="time" 0.714s | accuracy 22.0%

[12:38:05] [993/1000] x target="motorcycle" predicted="bike" 1.074s | accuracy 22.0%

[12:38:06] [994/1000] x target="maps" predicted="table" 1.015s | accuracy 21.9%

[12:38:07] [995/1000] x target="gavin" predicted="opening" 1.03s | accuracy 21.9%

[12:38:08] [996/1000] x target="intrigued" predicted="curious" 0.74s | accuracy 21.9%

[12:38:08] [997/1000] x target="makeover" predicted="little" 0.818s | accuracy 21.9%

[12:38:09] [998/1000] ✓ target="zane" predicted="zane" 0.733s | accuracy 21.9%

[12:38:10] [999/1000] x target="mortal" predicted="dead" 0.71s | accuracy 21.9%

[12:38:11] [1000/1000] x target="leader" predicted="legend" 1.037s | accuracy 21.9%

[12:38:11] Done: Gemma-3-4B — accuracy 21.9% (219/1000), avg 1.198s, 0 error(s).

Done. Gemma-3-4B: 21.9% accuracy over 1000 samples, 1.198s average.

Metrics

Model	Developer	Params	Accuracy (%)	Correct	Total	Avg time (s)	Errors
Gemma-3-4B	Google	4B	21.9	219	1000	1.198	0
Llama-3.2-3B	Meta	3B	20.8	208	1000	0.577	0
Ministral-8B	Mistral AI	8B	38.1	381	1000	0.731	0

7.3 MMLU – Results

Latest full run: all 57 subjects x 5 questions = 285 questions per model, zero API errors.

Rank	Model	Accuracy (%)	STEM	Human.	Social	Other	Reasoning consist. (%)	Avg words	Avg time (s)	Composite
1	Ministral-8B	78.9	78.9	73.9	83.3	80.0	69.1	54.1	1.537	0.731
2	Llama-3.2-3B	55.8	43.3	63.1	63.3	58.6	59.7	61.8	0.768	0.630
3	Gemma-3-4B	63.9	61.1	61.5	73.3	61.4	64.6	59.3	2.502	0.590

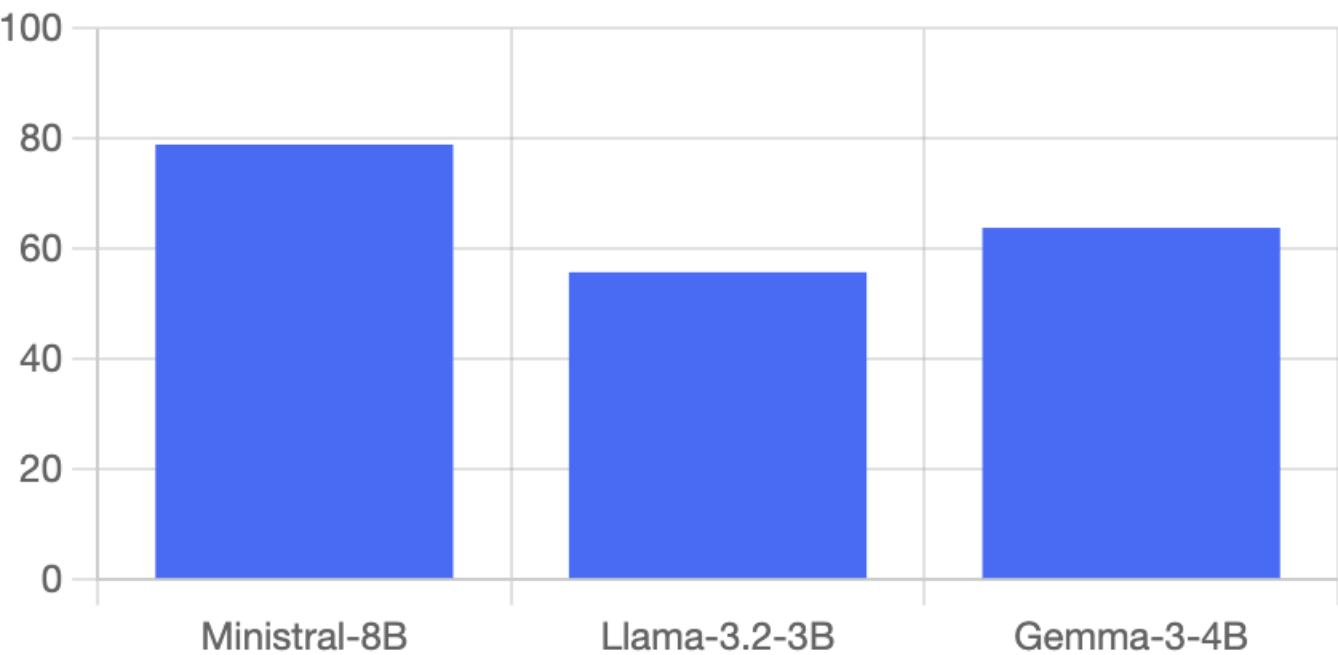
- Ministral-8B leads every category and the composite score.
- Gemma-3-4B is second on accuracy and reasoning consistency, but its 2.5 s average latency drops it below the much faster Llama-3.2-3B on the composite ranking.

Models are ranked per dimension (1 = best) on accuracy, speed, and reasoning consistency, then ordered by the composite score. Accuracy dominates the composite (70%); reasoning quality and latency (15% each) break ties, which mirrors how small language models are picked in practice: quality first, then cost and latency. See Section 6.5 for the fully worked composite-score calculation.

Charts

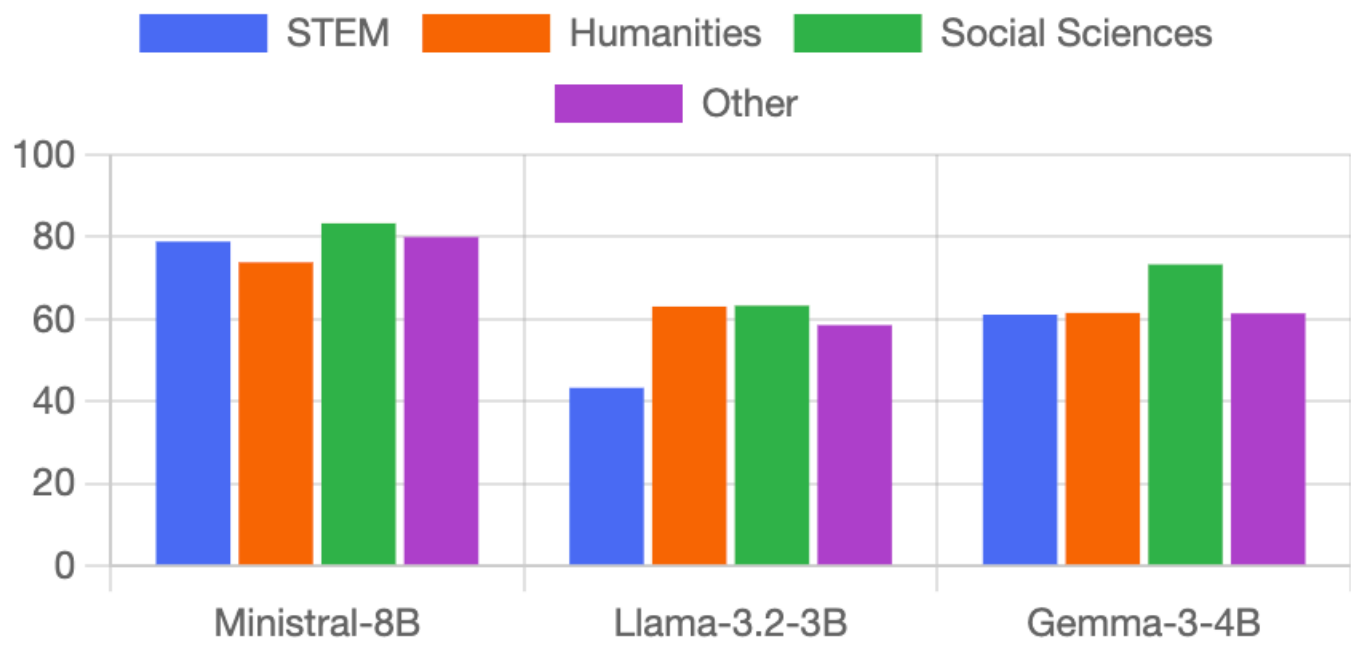
Overall accuracy per model, driving the composite ranking:

Accuracy by model



Subject-category accuracy (STEM / Humanities / Social Sciences / Other) per model:

Accuracy by category



7.4 MMLU — Web App

The MMLU run panel: model and questions-per-subject inputs, subject picker with category presets, and the live terminal that streams processing output below the input fields before and during a run.

MMLU Benchmark - Knowledge & Reasoning

Multiple-choice questions across the 57 MMLU subjects, fetched live from the free Hugging Face datasets-server API. Each model must reason step by step before answering; the reasoning is parsed, scored for consistency, and shown per question below.

Model

Gemma-3-4B

Questions / subject (1-50)

3

Run MMLU

Decoding

Subjects (18 selected)

All 57

STEM

Humanities

Social Sciences

Other

Clear

☒ STEM (18)

☒ abstract algebra

☒ astronomy

☒ college biology

☒ college chemistry

☒ college computer science

☒ college mathematics

☒ college physics

☐ Humanities (13)

☐ formal logic

☐ high school european history

☐ high school us history

☐ high school world history

☐ international law

☐ jurisprudence

☐ logical fallacies

☐ Social Sciences (12)

☐ econometrics

☐ high school geography

☐ high school government and politics

☐ high school macroeconomics

☐ high school microeconomics

☐ high school psychology

☐ Other (14)

☐ anatomy

☐ business ethics

☐ clinical knowledge

☐ college medicine

☐ global facts

☐ human aging

☐ management

mmlu — benchmark output

\$ Ready. Pick subjects above and press Run MMLU — live processing output streams here.

The Q/A section lists every question the selected model saw, grouped per subject with a correct-count badge and category tag:

Questions, answers & model reasoning

Every question the selected model saw, its pick vs the correct answer, and the parsed reasoning behind it.

Gemma-3-4B

Load Q/A

Gemma-3-4B - 57 subject(s), 285 question(s). Reasoning given on 100.0% of answers, consistent with the chosen option on 64.6%.

Abstract Algebra	2/5 correct	STEM	
Anatomy	5/5 correct	Other	
Astronomy	4/5 correct	STEM	
Business Ethics	4/5 correct	Other	
Clinical Knowledge	3/5 correct	Other	
College Biology	5/5 correct	STEM	
College Chemistry	3/5 correct	STEM	
College Computer Science	3/5 correct	STEM	
College Mathematics	1/5 correct	STEM	
College Medicine	2/5 correct	Other	
College Physics	3/5 correct	STEM	
Computer Security	3/5 correct	STEM	

Expanding a subject shows each question with the model's pick vs the correct answer, its parsed reasoning, the reasoning verdict, and per-question latency:

Questions, answers & model reasoning

Every question the selected model saw, its pick vs the correct answer, and the parsed reasoning behind it.

Gemma-3-4B ▾

Load Q/A

Gemma-3-4B - 57 subject(s), 285 question(s). Reasoning given on 100.0% of answers, consistent with the chosen option on 64.6%.

Abstract Algebra 2/5 correct STEM

Q1. Find the degree for the given field extension $\mathbb{Q}(\sqrt{2}, \sqrt{3}, \sqrt{18})$ over $\mathbb{Q}$.

correct Correct, weakly supported 4.547s

A. 0

B. 4 correct answer model's pick

C. 2

D. 6

Model reasoning:

We are looking for the degree of the field extension $\mathbb{Q}(\sqrt{2}, \sqrt{3}, \sqrt{18})$ over $\mathbb{Q}$. First, note that $\sqrt{18} = 3\sqrt{2}$, so $\mathbb{Q}(\sqrt{2}, \sqrt{3}, \sqrt{18}) = \mathbb{Q}(\sqrt{2}, \sqrt{3})$. The extension $\mathbb{Q}(\sqrt{2})$ over $\mathbb{Q}$ has degree 2, and the extension $\mathbb{Q}(\sqrt{2}, \sqrt{3})$ over $\mathbb{Q}(\sqrt{2})$ has degree 2. Therefore, the degree of the entire extension is $2 + 2 = 4$.

Q2. Let $p = (1, 2, 5, 4)(2, 3)$ in S_5 . Find the index of $\langle p \rangle$ in S_5 .

correct Correct, weakly supported 8.615s

A. 8

B. 2

C. 24 correct answer model's pick

D. 120

Model reasoning:

First, we need to find the cycle type of the permutation $p = (1, 2, 5, 4)(2, 3)$. Applying the permutations from right to left, we have $(1, 2, 5, 4)(2, 3) = (1, 2, 5, 4)(2, 3) = (1, 2, 3, 5, 4)$. This is a 5-cycle. The order of a cycle of length n is n . Therefore, the order of p is 5. The index of the cyclic subgroup generated by p , denoted by $\langle p \rangle$, in the symmetric group S_5 is given by the order of the element p divided by their greatest common divisor. Since the order of p is 5, the index of $\langle p \rangle$ in S_5 is 5. However, this is not one of the answer choices. Let's recompute the permutation: $(1, 2, 5, 4)(2, 3) = (1, 2, 5, 4)(2, 3) = (1, 2, 3, 5, 4)$. The cycle decomposition is $(1)(2, 3, 5, 4)$. The order of the permutation is 5. The number of elements in S_5 is $5! = 120$. The index of $\langle p \rangle$ in S_5 is $120/5 = 24$.

8. Discussion of the Results

8.1 Why These LAMBADA Results?

- Accuracy gap (38.1% Ministral vs. 20.8–21.9% for the other two) matches the compression story: Ministral is the only model not distilled or pruned from a larger checkpoint. LAMBADA specifically punishes compression, because the correct word is very often a rare proper noun or specific detail that a distilled/pruned model is more likely to have smoothed over.
- LAMBADA gives no multiple-choice options — the model must generate the exact right token from the full vocabulary with nothing to recognize from, stressing raw language-modeling precision more than instruction-following, which is exactly where heavy pruning tends to cost the most.
- Gemma-3-4B's latency (1.198s, more than double Llama's 0.577s) is not primarily an architecture story here: this evaluation's own retry log shows Gemma's run hit a sustained string of HTTP 429 rate-limit responses from its OpenRouter-hosted provider, each costing up to ~60s in retries before succeeding — inflating its measured average well above its likely "clean" speed (~0.58s, based on an earlier 50-sample dry run before that congestion).
- All three models finished at 0 recorded errors — the retry-with-backoff logic added after that congestion was diagnosed always eventually succeeded rather than giving up.

8.2 Why These MMLU Results?

- Ministral-8B's win here is even larger than on LAMBADA (78.9% vs. runner-up 63.9%): MMLU tests knowledge breadth across 57 subjects, and raw parameter count (8B vs. 3–4B) tracks especially closely with how much factual knowledge a model can store — more so than with pure language-modeling fluency.
- Category accuracy reveals unevenness the headline number hides: Llama-3.2-3B swings from 43.3% (STEM) to 63.3% (Social Sciences) — a 20-point gap between its weakest and strongest domains. Ministral-8B's own spread is much tighter (73.9%–83.3%, a 9.4-point gap) — evidence of more consistent competence, not just a higher average.

- Reasoning length is not reasoning quality: Ministral writes the shortest average explanation (54.1 words) yet the most consistent one (69.1%); Llama writes the longest (61.8 words) yet the least consistent (59.7%) — verbosity doesn't buy correctness.
- The composite ranking keeps Llama-3.2-3B ahead of Gemma-3-4B (0.630 vs. 0.590) despite Gemma's higher raw accuracy (63.9% vs. 55.8%) — because Gemma's 2.502s average latency is more than 3x Llama's, and speed carries 15% of the score. This is the clearest illustration in this study of how weighting can reorder a leaderboard relative to accuracy alone.

8.3 Comments and Discussions

Accuracy & knowledge depth

Ministral-8B leads both benchmarks (38.1% LAMBADA, 78.9% MMLU) — the only model not distilled or pruned from a larger checkpoint, so it keeps more raw capacity for precise recall. Gemma-3-4B and Llama-3.2-3B are both compressed from bigger teacher/checkpoint models — a cost that shows up most on exact recall of rare words or facts.

Speed & latency

Llama-3.2-3B is fastest on both benchmarks (0.577s LAMBADA, 0.768s MMLU) — the smallest, most compressed model. Gemma-3-4B is the slowest despite being smaller than Ministral-8B — interleaved global-attention layers, fewer OpenRouter hosting providers, and (on its LAMBADA run specifically) transient provider rate-limit congestion all inflate its latency.

Reasoning quality (MMLU)

Ministral-8B has the highest reasoning consistency (69.1%); Gemma-3-4B is close behind (64.6%) despite its latency cost; Llama-3.2-3B is lowest (59.7%) despite being fastest. Speed, accuracy, and reasoning quality don't move together — the composite score (70/15/15) exists precisely to weigh that trade-off.

Use-case perspective

- Ministral-8B: best when accuracy and reasoning quality matter most and an 8B footprint is affordable.
- Llama-3.2-3B: best for latency-sensitive or edge deployment.
- Gemma-3-4B: strong quality-per-parameter on paper, but the weakest speed/cost trade-off in this specific hosted (OpenRouter) setup.

9. Conclusions

Does a larger, non-distilled model beat smaller, compressed ones across both benchmarks?

- Yes — in raw accuracy and knowledge depth: Ministral-8B wins LAMBADA (38.1%) and MMLU (78.9%).
- No — in latency and cost-efficiency: Llama-3.2-3B, the smallest and most compressed model, is consistently the fastest.

For this evaluation:

- Ministral-8B is the strongest choice when accuracy and reasoning quality matter most.
- Llama-3.2-3B is preferable for latency-sensitive or edge deployment.
- Gemma-3-4B sits in between — competitive in quality-per-parameter terms, but currently the weakest on measured speed via this hosted setup.
- LAMBADA rewards real use of context; MMLU rewards breadth of knowledge and reasoning that actually supports the answer — together they separate raw language modeling from broader competence.

- Paperno, D. et al. (2016). The LAMBADA dataset. Proceedings of ACL 2016.
- Hendrycks, D. et al. (2021). Measuring Massive Multitask Language Understanding. Proceedings of ICLR 2021.
- Google (2025). Gemma 3 technical report.
- Meta (2024). Llama 3.2 model card.
- Mistral AI (2024). Ministral model family.